

E9: 309 ADL 30-12-2020



Housekeeping

✳ Midterm project II presentations

- ➡ Remaining presentations (31-12-2020) Starting from 430pm

- ➡ Same format as previous evaluation

✳ Midterm project III

- ➡ Abstract submission deadline (Jan 10th)

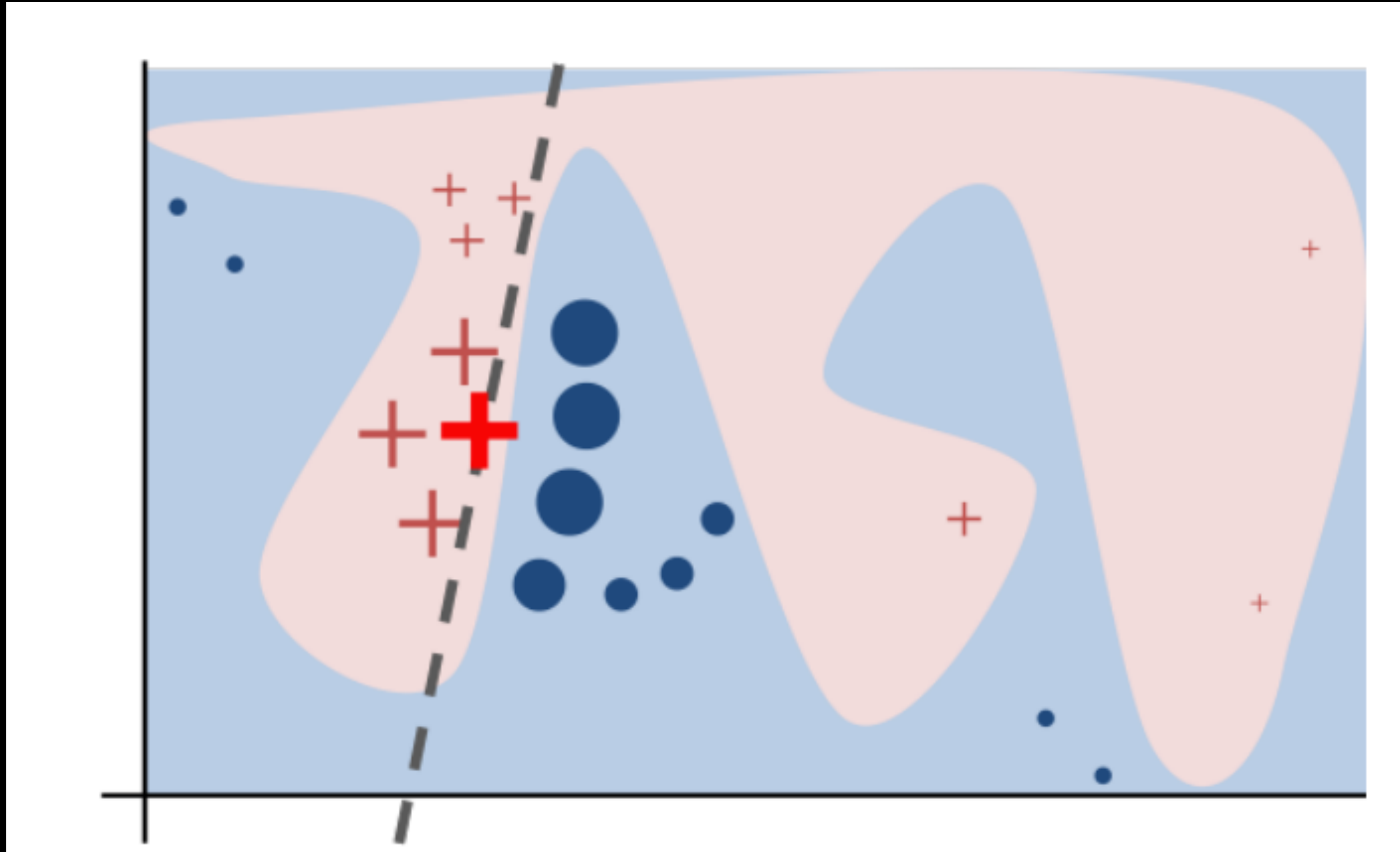
- ✓ Evaluation after final exam (1st week of Feb)

✳ Final Exam (as per IISc schedule)

- ✓ Jan 23rd afternoon!

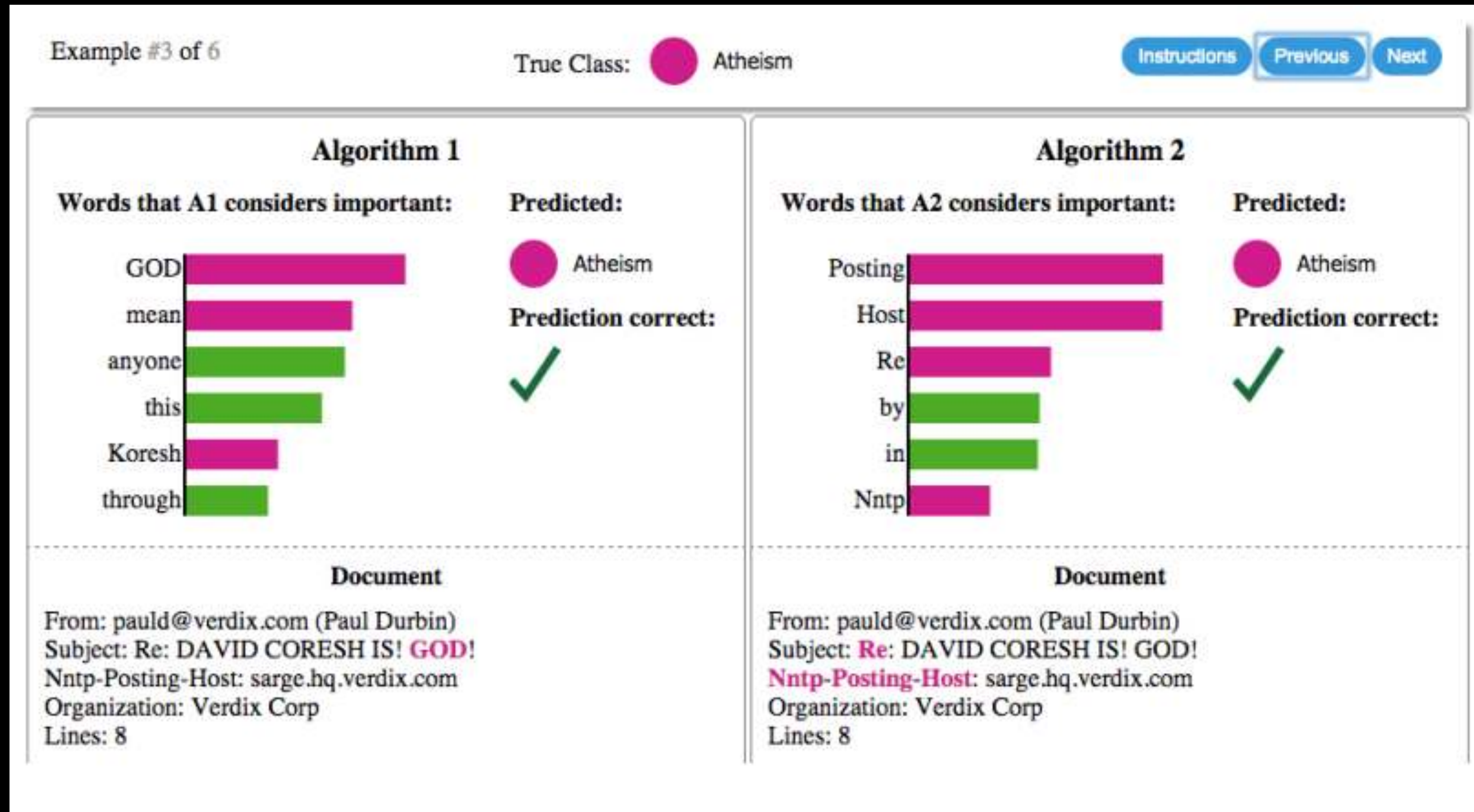


Knowledge distillation for explainability



LIME model - text example

✳ Building sparse linear regression for each output class

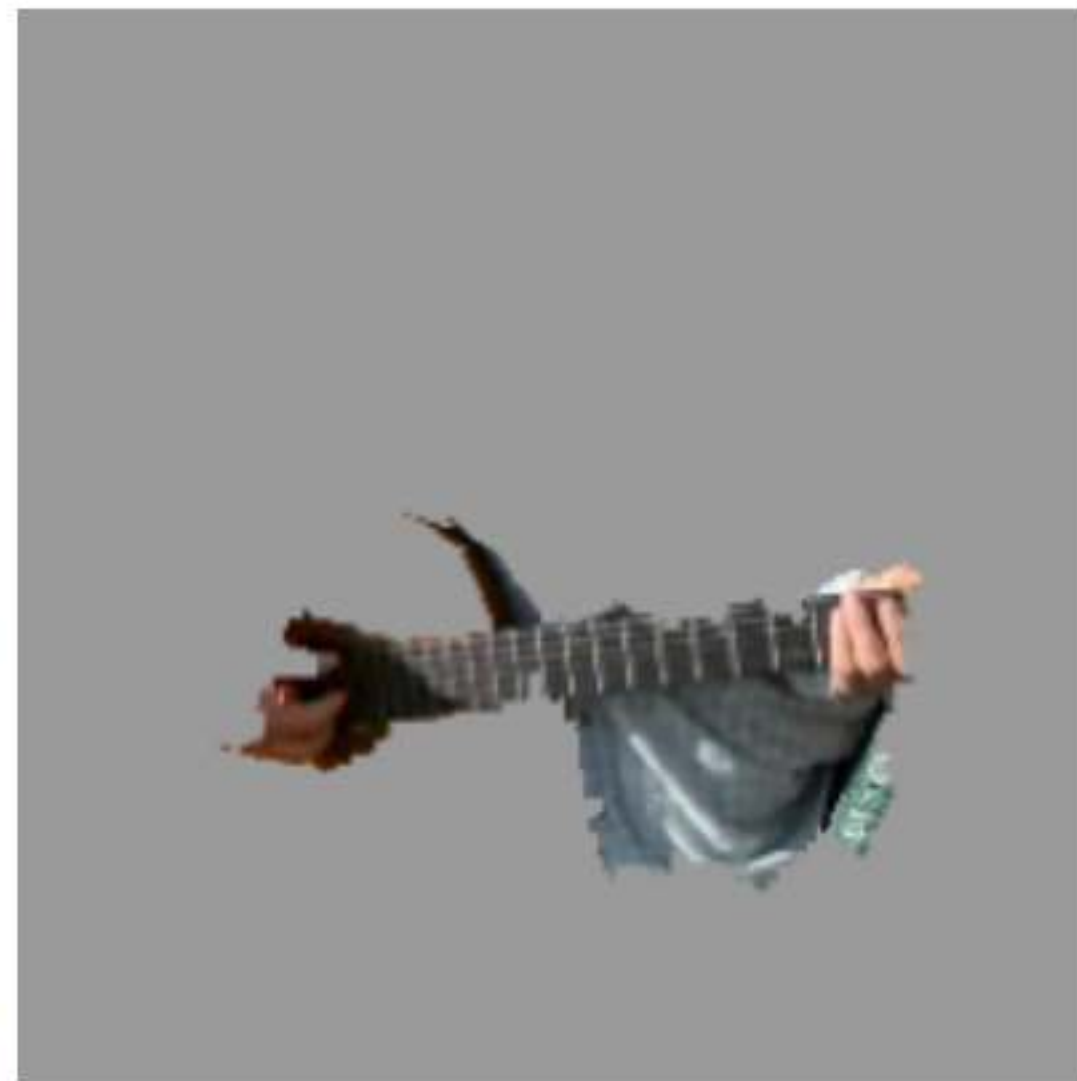


LIME model - Image example

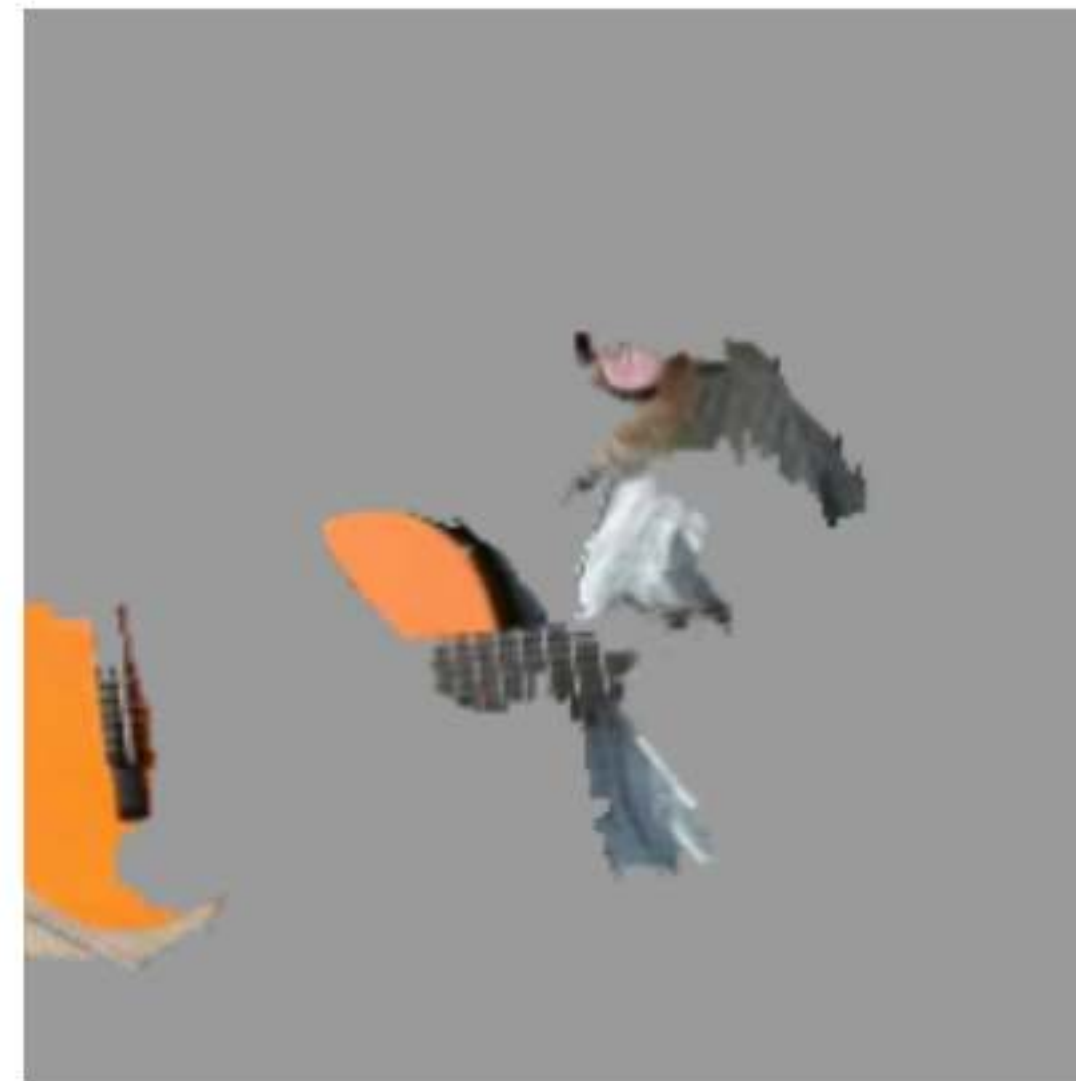
✳ Building sparse linear regression for each output class



(a) Original Image



(b) Explaining *Electric guitar*



(c) Explaining *Acoustic guitar*



(d) Explaining *Labrador*

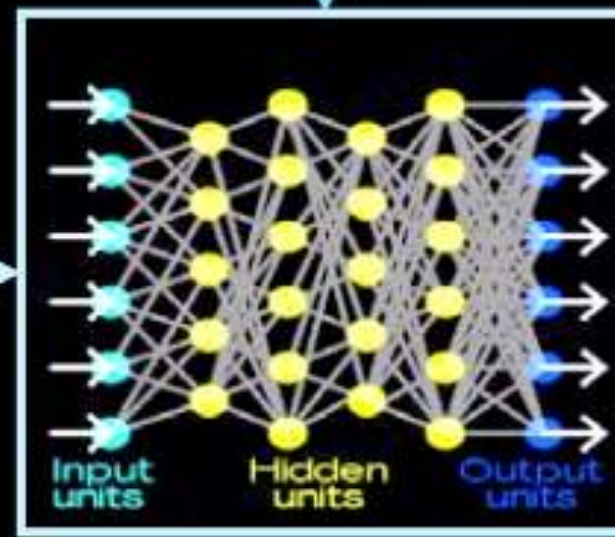
Future Research Directions

Today



Training Data

Learning Process



Learned Function

This is a cat
(p = .93)

Output



User with a Task

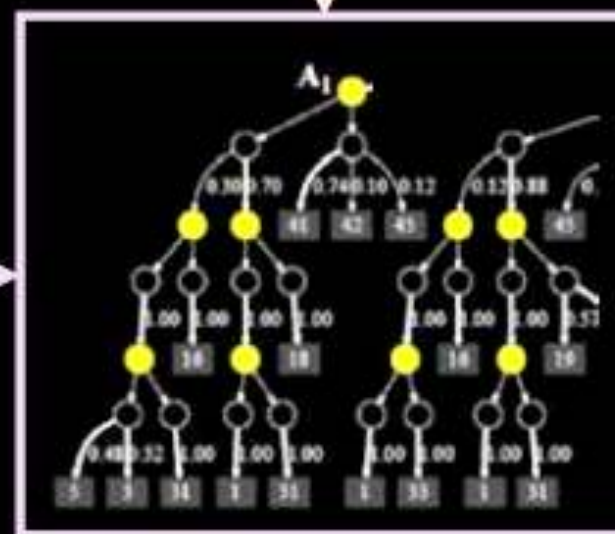
- Why did you do that? ✓
- Why not something else? ✓
- When do you succeed? ✓
- When do you fail? ✓
- When can I trust you? ✓
- How do I correct an error? ✓

Tomorrow



Training Data

New Learning Process

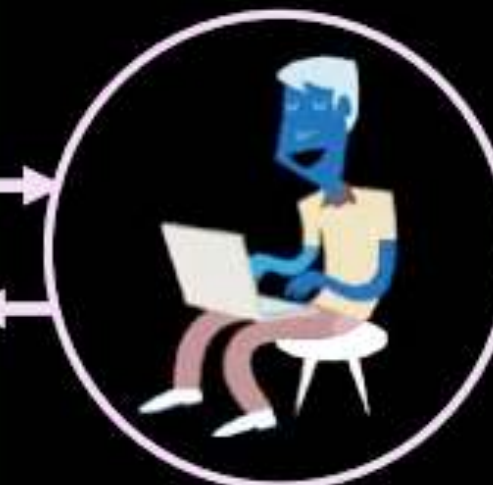


Explainable Model

This is a cat:
• It has fur, whiskers, and claws.
• It has this feature:



Explanation Interface

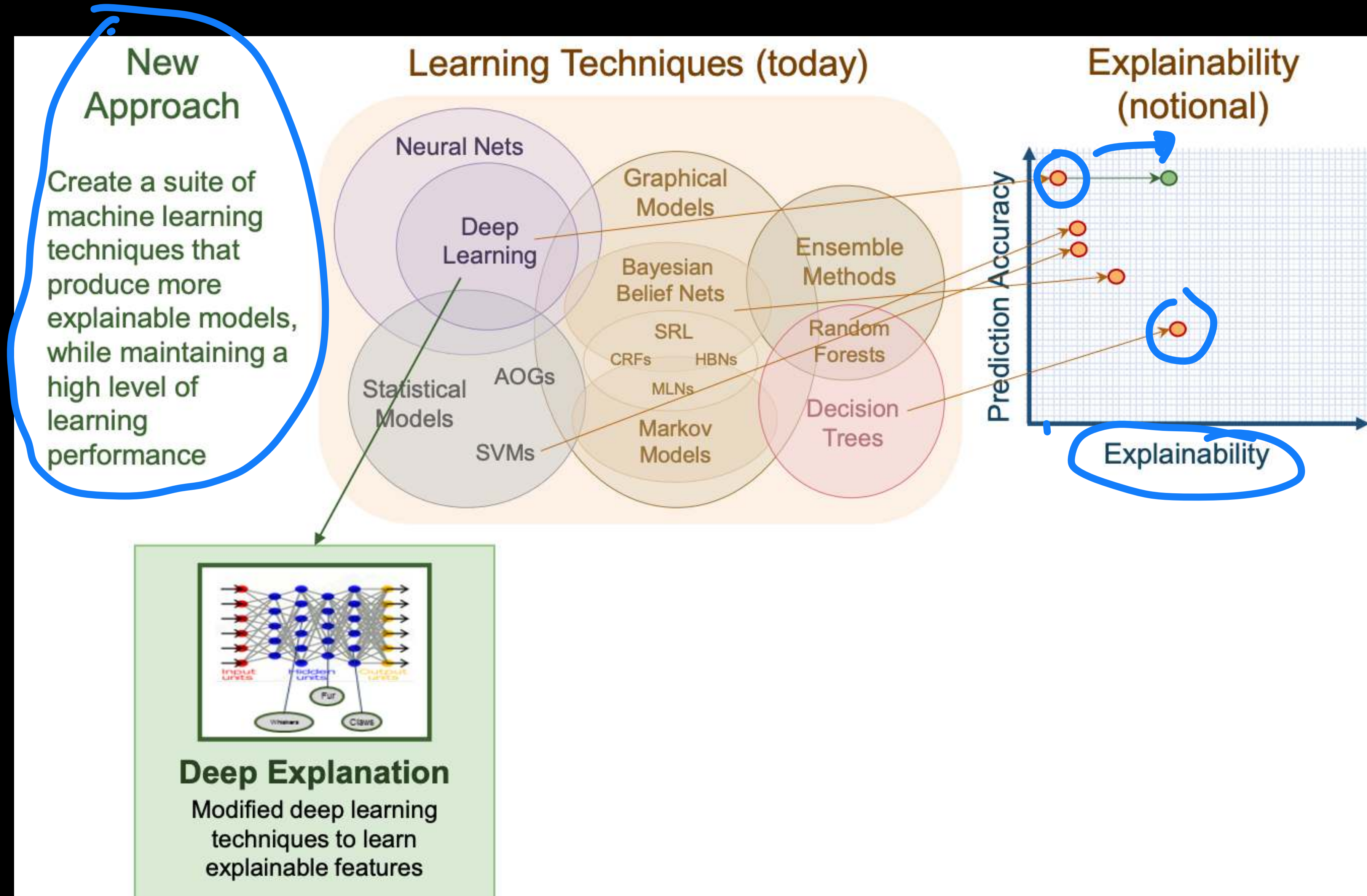


User with a Task

- I understand why
- I understand why not
- I know when you'll succeed
- I know when you'll fail
- I know when to trust you
- I know why you erred

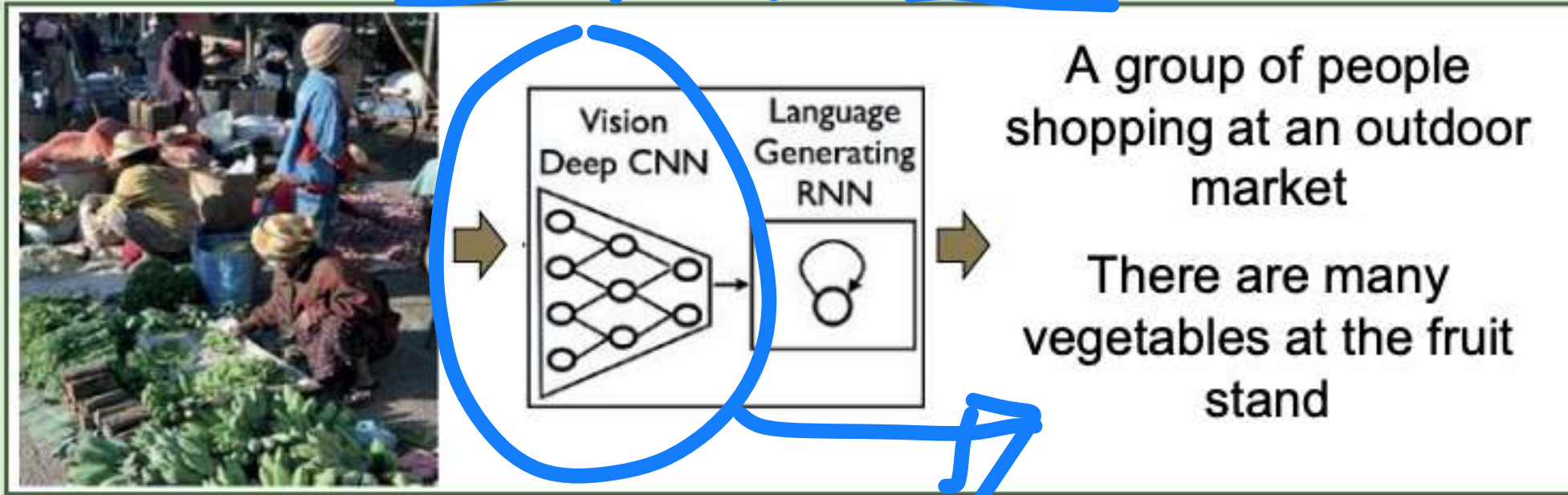
XAI

Future Research Directions



Future Research Directions

Generating Image Captions

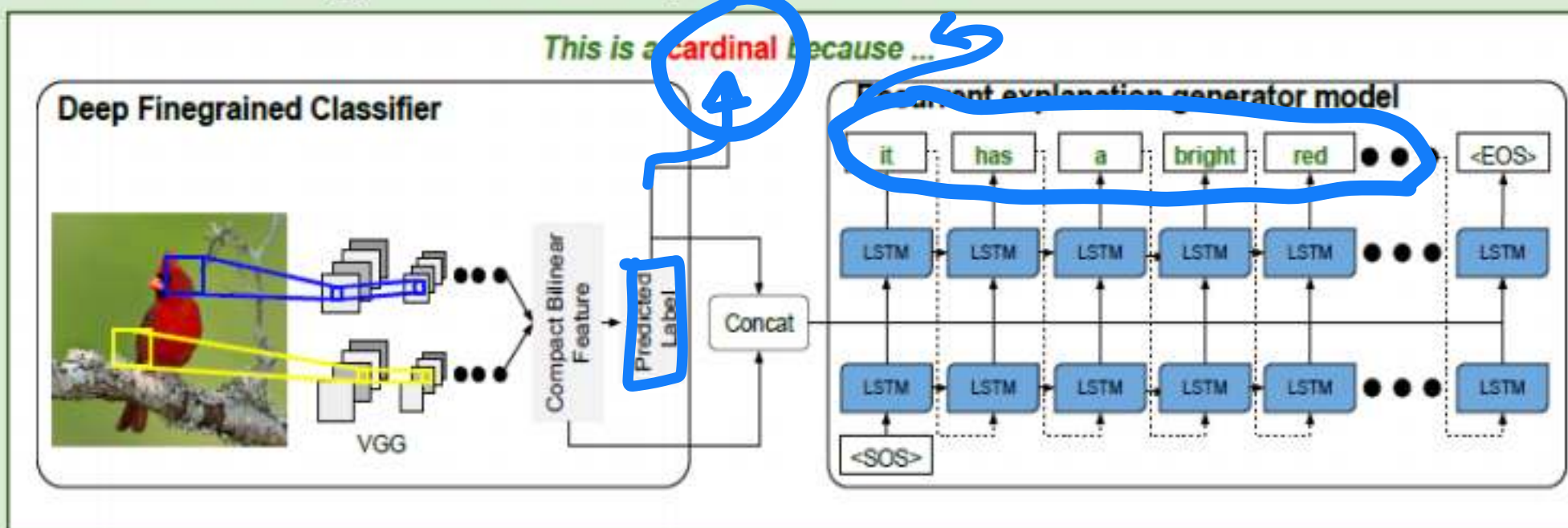


- A CNN is trained to recognize objects in images
- A language generating RNN is trained to translate features of the CNN into words and captions.

Example Explanations



Generating Visual Explanations



Researchers at UC Berkeley have recently extended this idea to generate explanations of bird classifications. The system learns to:

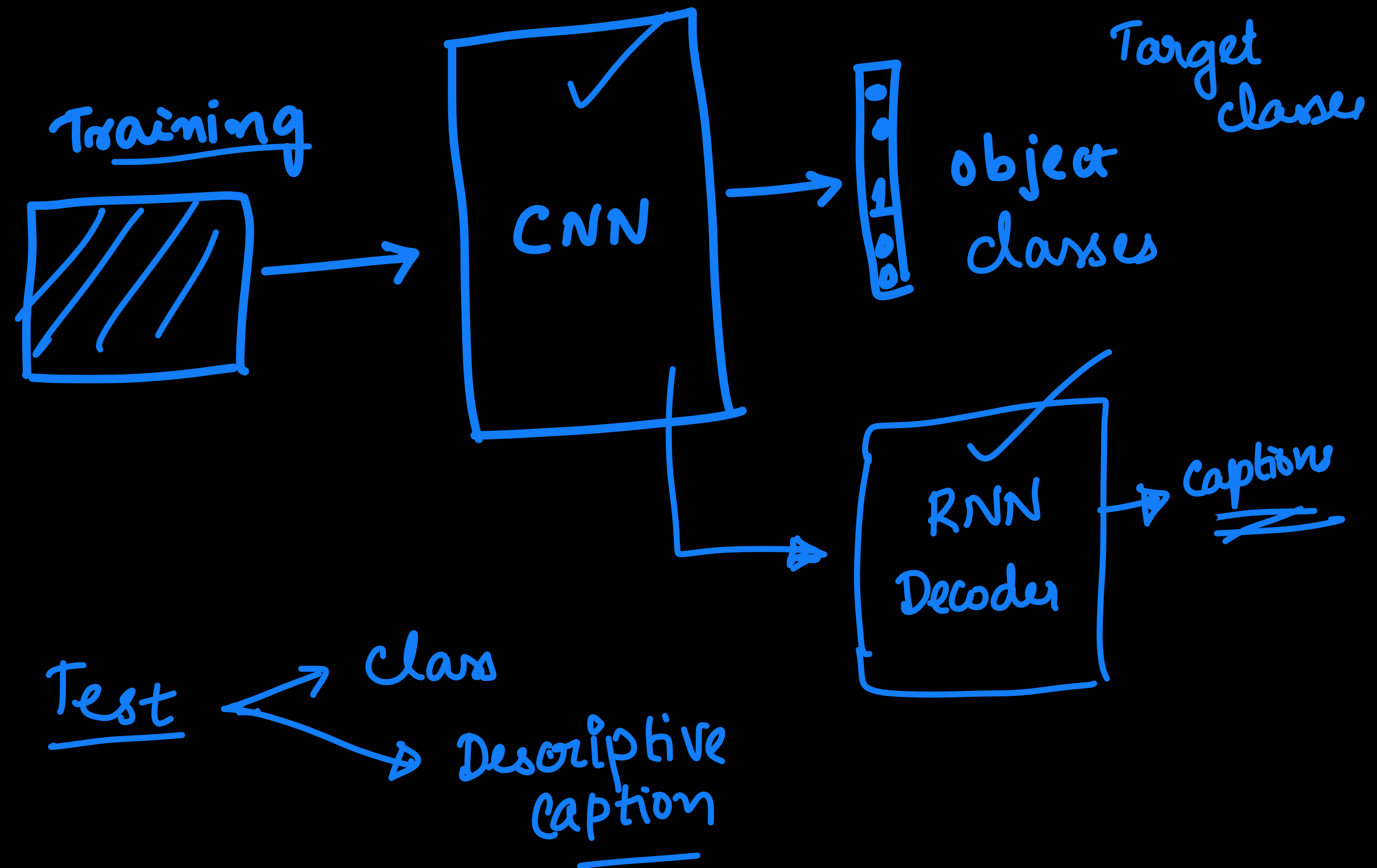
- Classify bird species with 85% accuracy
- Associate *image descriptions* (discriminative features of the image) with *class definitions* (image-independent discriminative features of the class)

Limitations

- Limited (indirect at best) explanation of internal logic
- Limited utility for understanding classification errors

Hendricks, L.A, Akata, Z., Rohrbach, M., Donahue, J., Schiele, B., and Darrell, T. (2016). Generating Visual Explanations, arXiv:1603.08507v1 [cs.CV] 28 Mar 2016





Topics thus far ...

Visual and Time Series Modeling: Semantic Models, Recurrent neural models and LSTM models, Encoder-decoder models, Attention models.

Representation Learning, Causality And Explainability: t-SNE visualization, Hierarchical Representation, semantic embeddings, gradient and perturbation analysis, Topics in Explainable learning, Structural causal models.

Unsupervised Learning: Restricted Boltzmann Machines, Variational Autoencoders, Generative Adversarial Networks.

New Architectures: Capsule networks, End-to-end models, Transformer Networks.

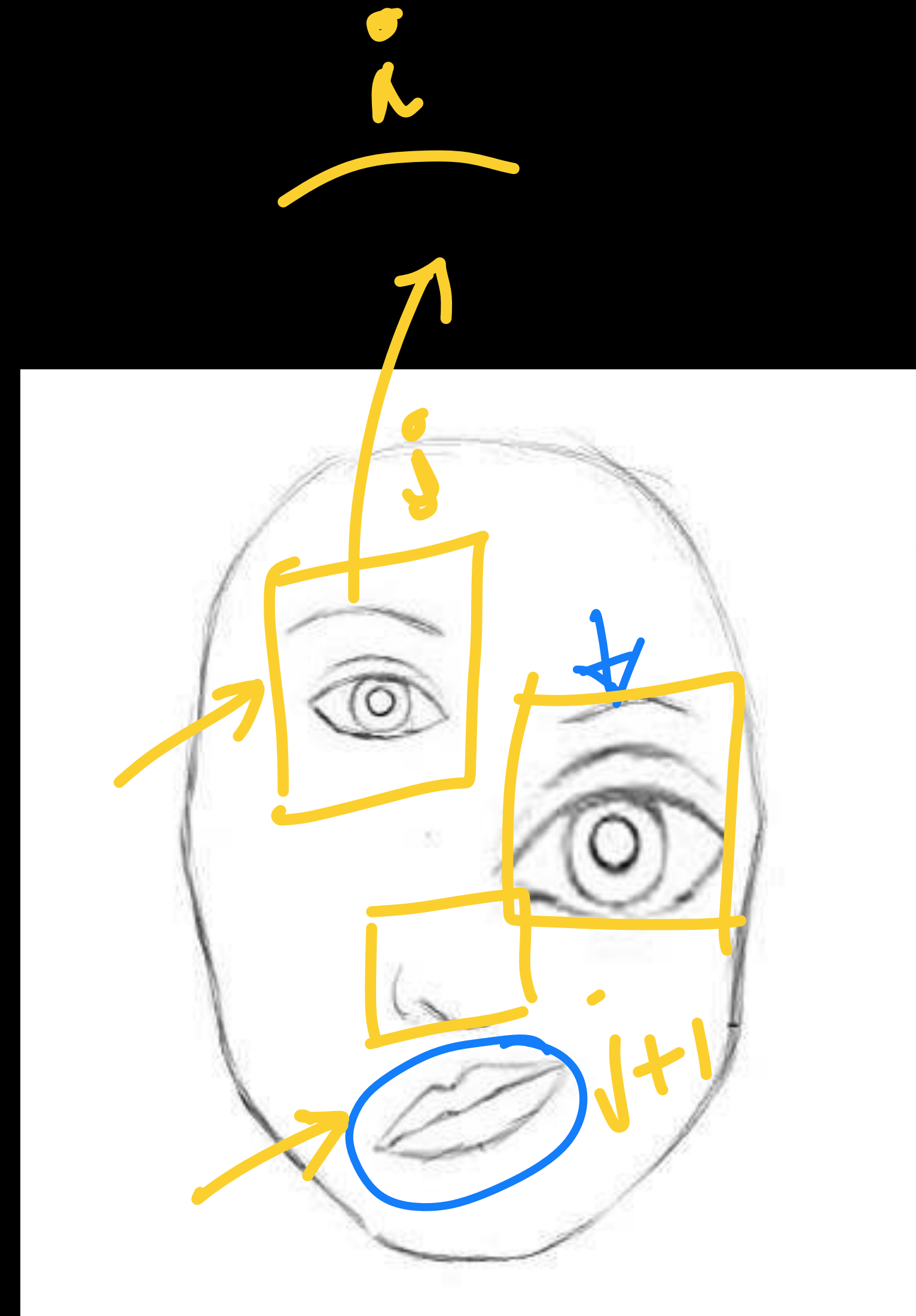
Applications: Applications in in NLP, Speech, Image/Video domains in all modules.



Problem with current deep learning networks

✱ Convolutional neural networks with filtering and max pooling layers

- ✓ Generate indicators of object parts
- ✓ check the presence of object parts and confirm the object identity.
- ✓ Often fails to understand the spatial relationship between object parts.



Is this a face ?

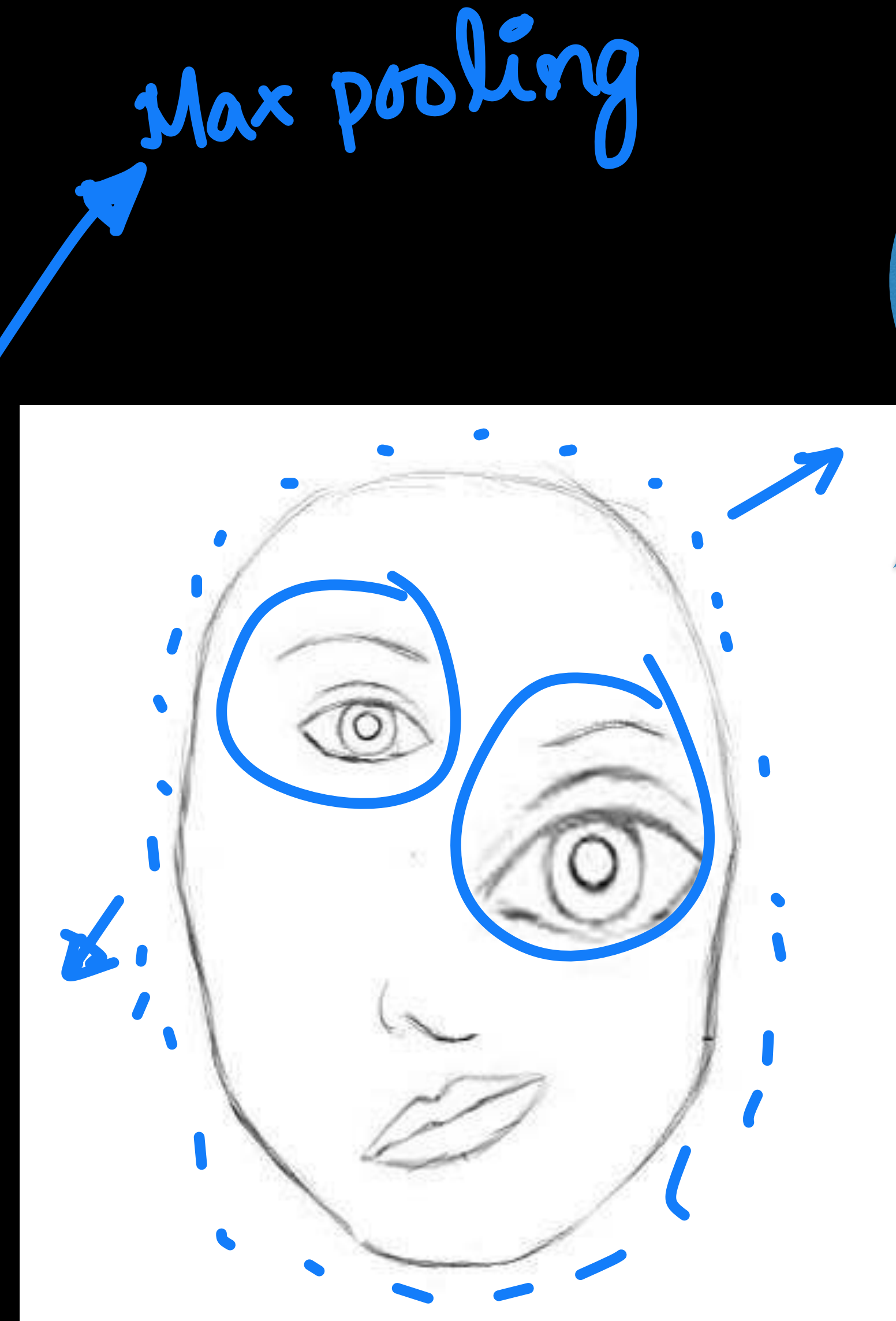
Problem with current deep learning networks

* Question

- Can we learn the presence of object parts and their implicit spatial information (position) ?

* Invariance versus equivariance

- Invariance - resistance to translations and shifts [CNNs]
- Equivariance - relationship of parts with other parts have to preserved.



Is this a face ?

Capsule networks

* Traditional networks

→ Weigh the inputs and generate a scalar output *at each neuron.*

* Key idea in capsule networks (neurons to capsules)

- ✓ Move individual neuron outputs from scalar to vector
- ✓ Encode the probability of presence of an attribute along the magnitude of the vector output
- ✓ Encode the pose (translation + rotation) in the angle of the vector output

*color, texture
Orientation*



Capsule Networks

Dynamic Routing Between Capsules

Sara Sabour

Nicholas Frosst

Geoffrey E. Hinton

Google Brain

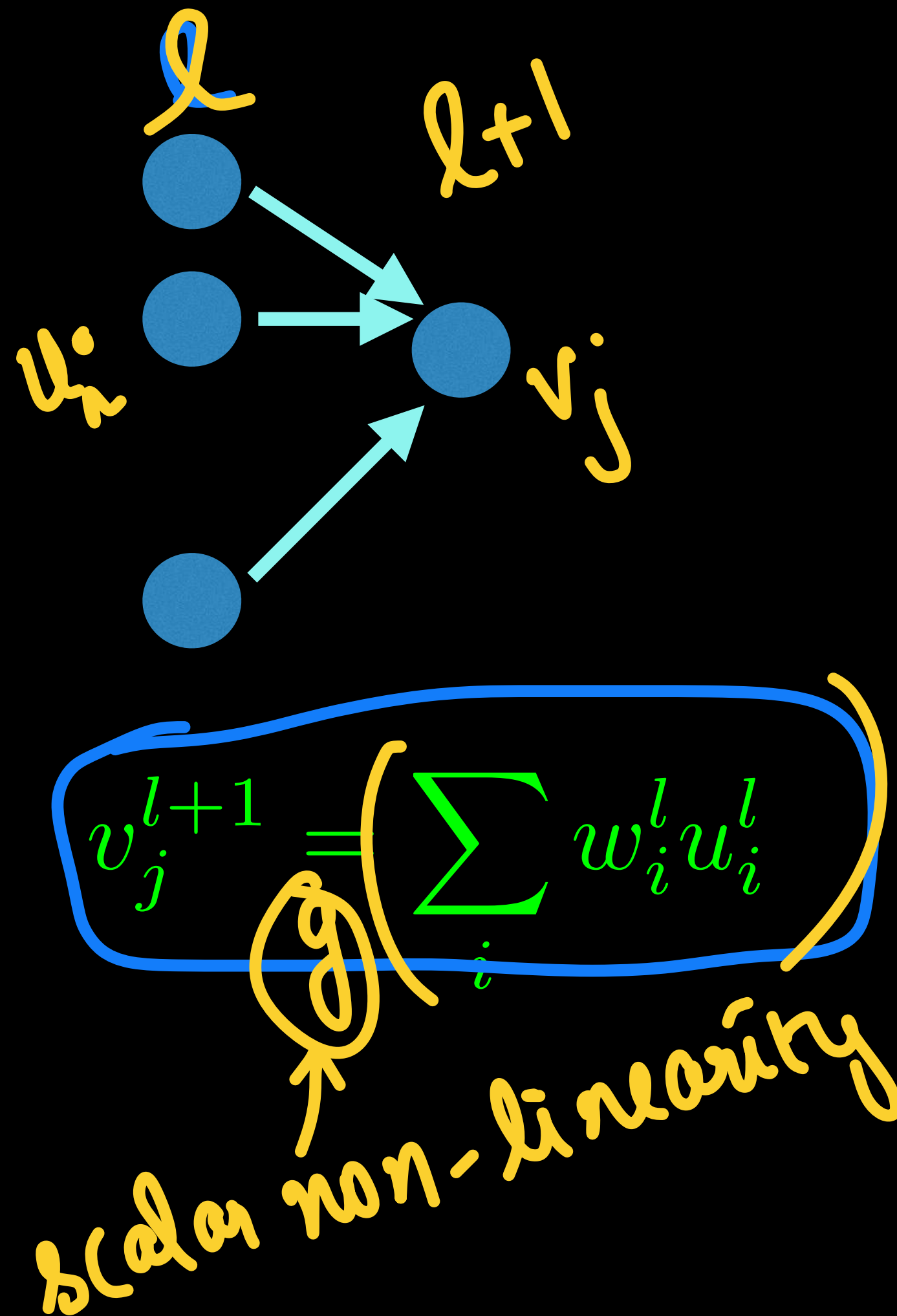
Toronto

`{sasabour, frosst, geoffhinton}@google.com`

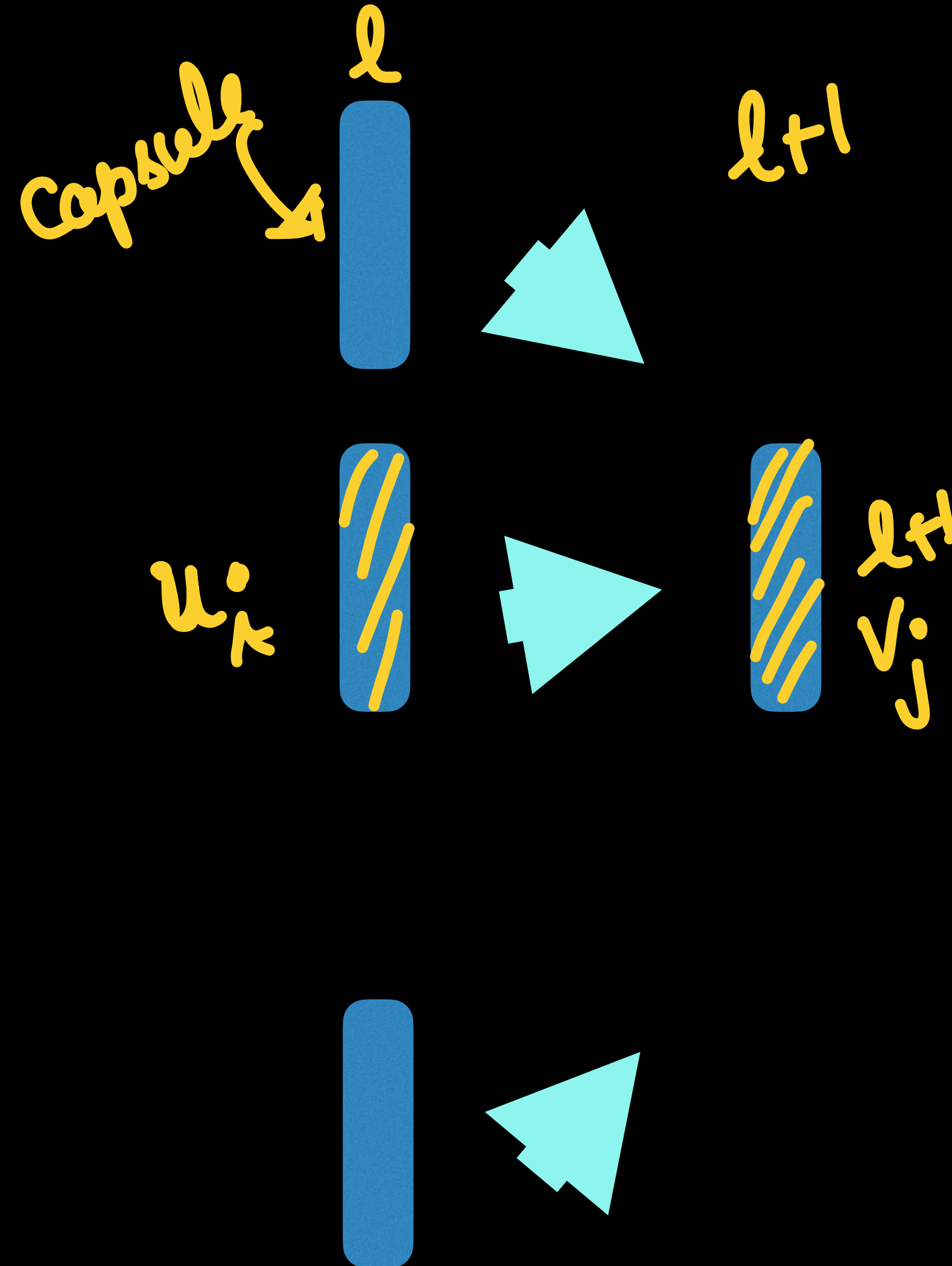


From a layer of neurons to layer of capsules

✧ Conventional neurons



✧ Capsule network

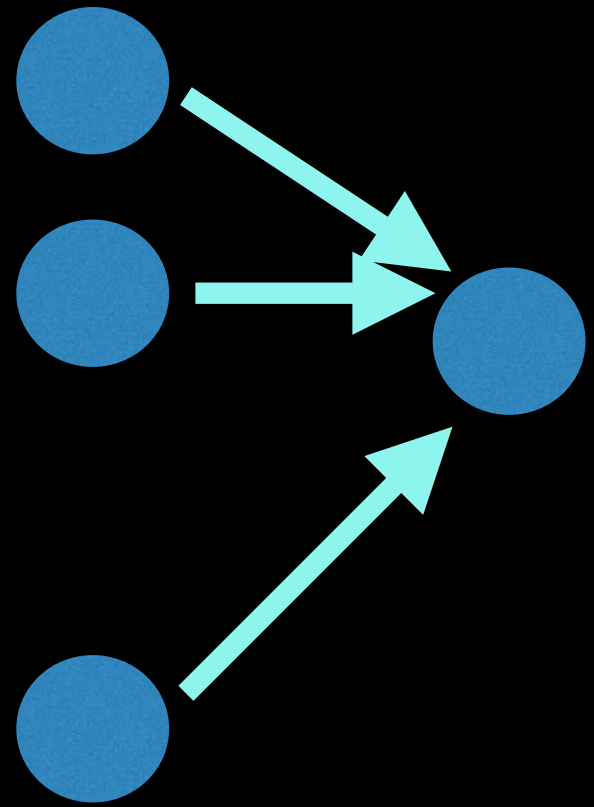


Prediction vector

$$\hat{\mathbf{u}}_{j|i}^{l+1} = \mathbf{W}_{ij} \mathbf{u}_i^l$$

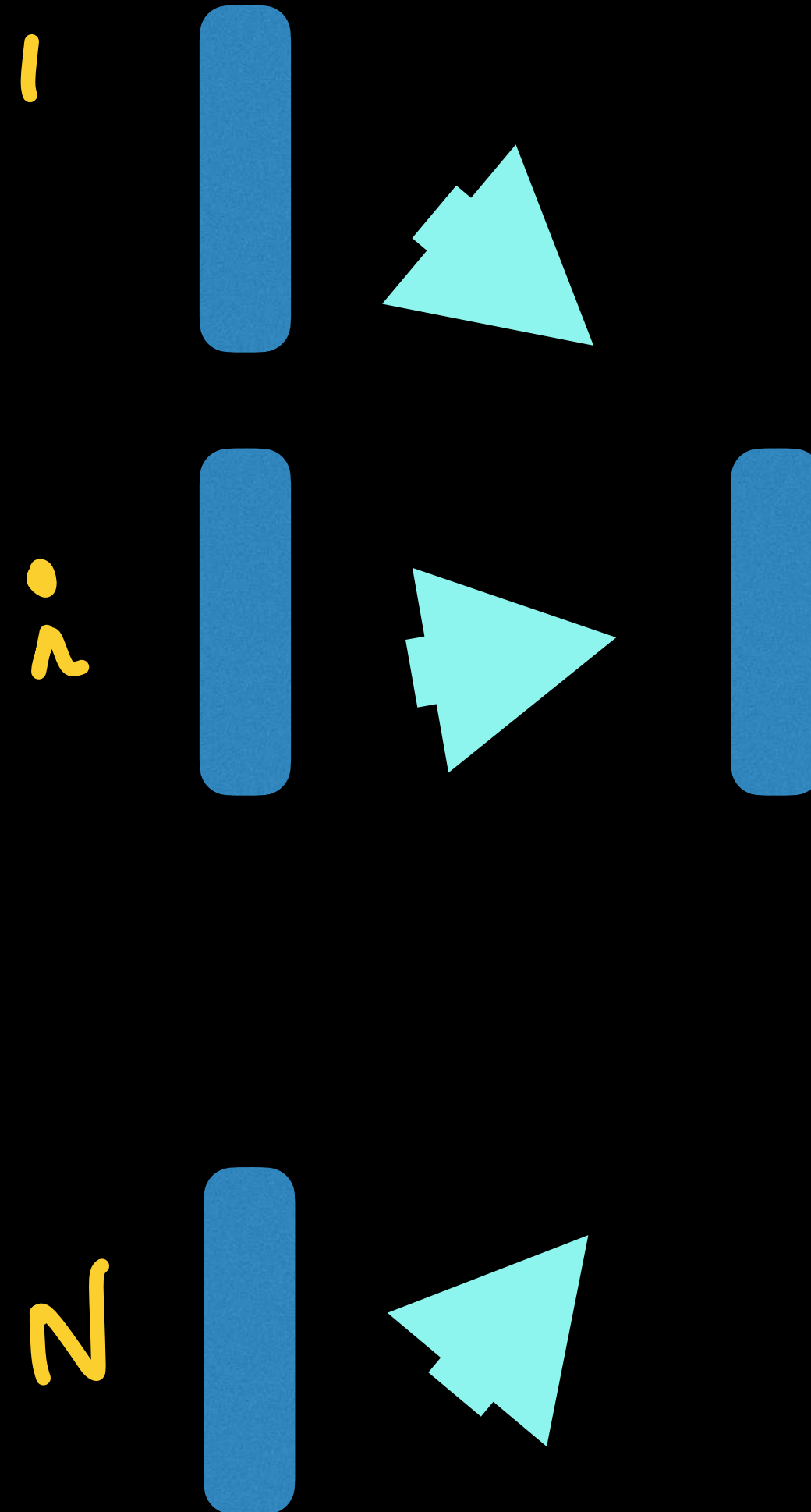
From a layer of neurons to layer of capsules

✧ Conventional neurons



$$v_j^{l+1} = \sum_i w_i^l u_i^l$$

✧ Capsule network



Prediction vector

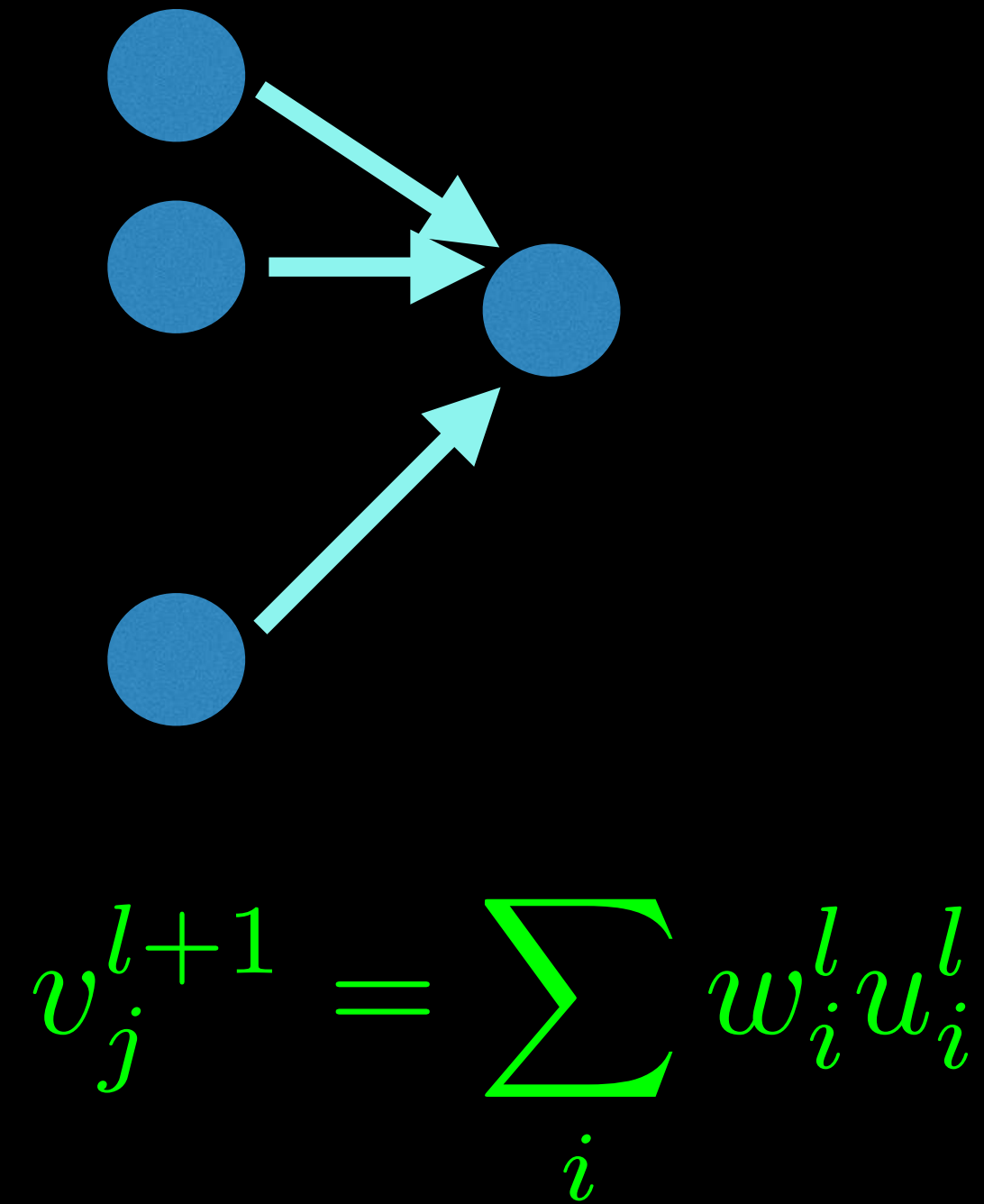
$$\hat{\mathbf{u}}_{j|i}^{l+1} = \mathbf{W}_{ij} \mathbf{u}_i^l$$

Coupling

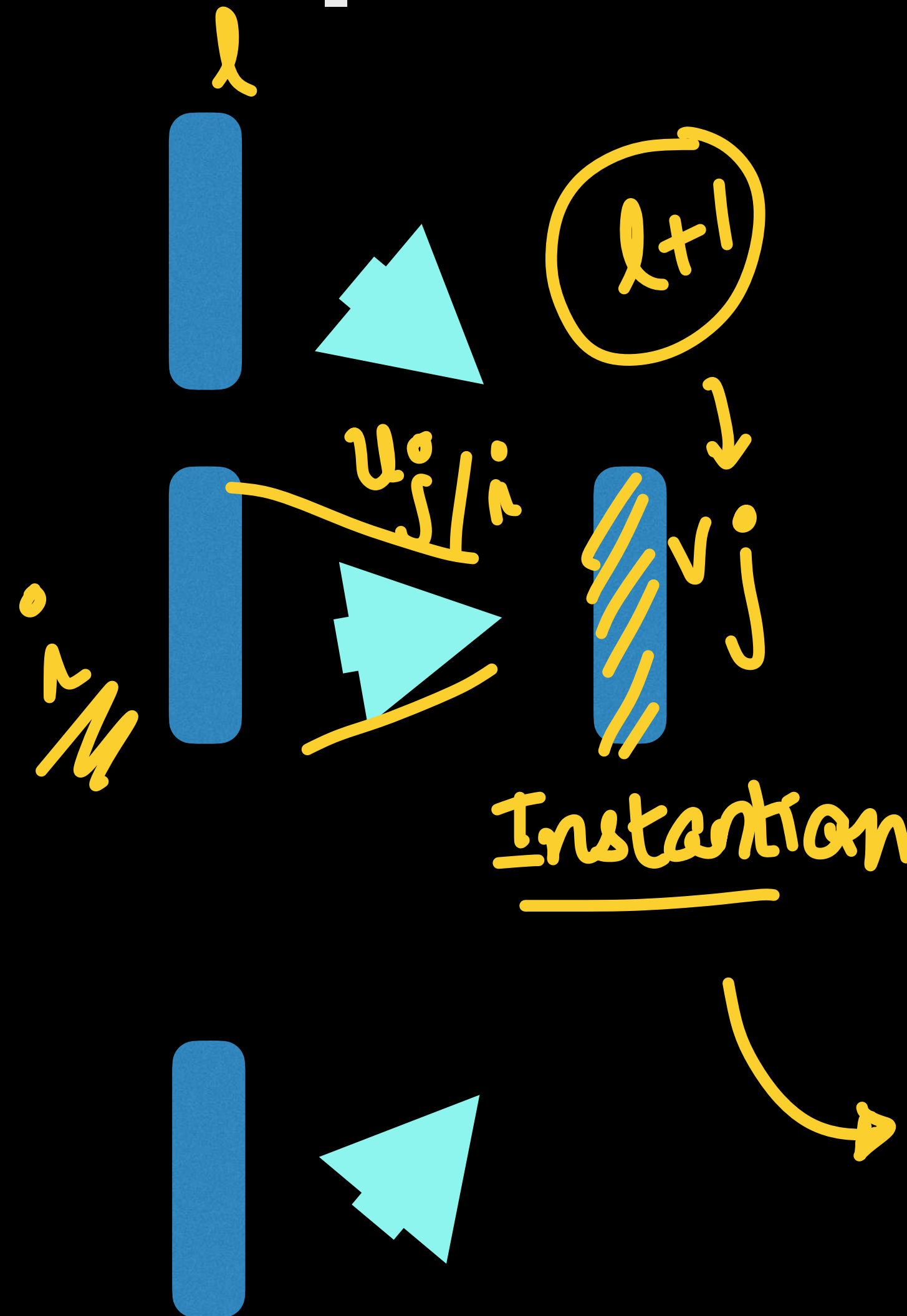
$$\mathbf{s}_j^{l+1} = \sum_i c_{ij} \hat{\mathbf{u}}_{j|i}^{l+1}$$

From a layer of neurons to layer of capsules

* Conventional neurons



* Capsule network



Prediction vector

$$\hat{\mathbf{u}}_{j|i}^{l+1} = \mathbf{W}_{ij} \mathbf{u}_i^l$$

Coupling

$$\mathbf{s}_j^{l+1} = \sum_i c_{ij} \hat{\mathbf{u}}_{j|i}^{l+1}$$

Instantiation parameter

$$\mathbf{v}_j^{l+1} = \text{Squash}(\mathbf{s}_j^{l+1})$$

Capsule network

* Squashing non-linearity

$v_j \leftarrow \text{Squash}(s_j^{l+1}) = \frac{\|s_j\|^2}{1 + \|s_j\|^2} \frac{s_j}{\|s_j\|}$

Handwritten notes:
 - $\text{mag} \rightarrow 0, 1$ (pointing to the denominator)
 - $\frac{\|s_j\|}{\|s_j\|} \rightarrow 1$ (pointing to the vector part)
 - $\frac{\|s_j\|}{\|s_j\|} \rightarrow 0$ (pointing to the vector part)

→ Makes the vector magnitude range from (0,1)

✓ Interpretation of the length of the vector as a probability of the presence of a particular object part

→ Preserves the direction of the original vector.

Capsule network

* Coupling

$$c_{ij} = \frac{e^{b_{ij}}}{\sum_k e^{b_{ik}}}$$

softmax

$$s_j = \sum_i c_{ij} \underline{u_{j/i}} \quad i \leftrightarrow j \quad l \leftrightarrow l+1$$

* The coefficients

$$b_{ij} \leftarrow b_{ij} + \underline{u_{j/i}^T v_j}$$

b_{ij}

- ✓ Initial values indicate prior probabilities that i should be coupled with j
- ✓ Depend on the location and type of the capsule

Capsule network

* Coupling

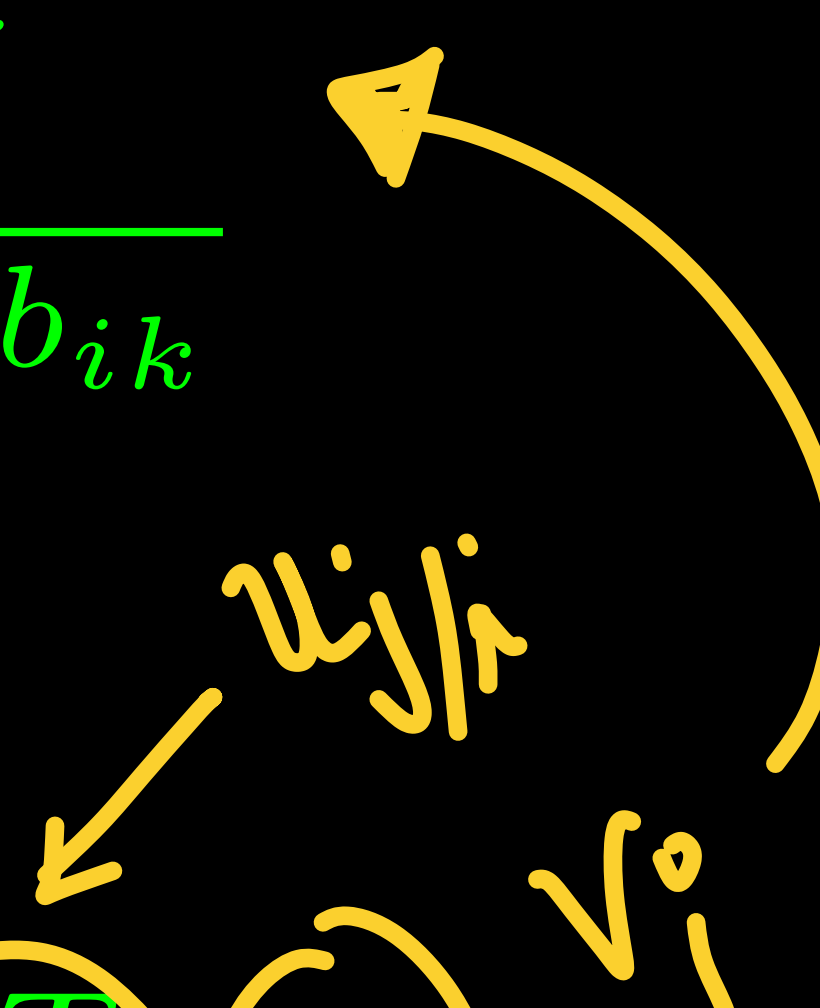
$$c_{ij} = \frac{e^{b_{ij}}}{\sum_k e^{b_{ik}}}$$


Diagram illustrating the coupling coefficient c_{ij} and the bias update b_{ij} . The coupling coefficient is defined as $c_{ij} = \frac{e^{b_{ij}}}{\sum_k e^{b_{ik}}}$. The bias update is $b_{ij} \leftarrow b_{ij} + \mathbf{u}_{j|i}^T \mathbf{v}_j$. The diagram shows a yellow arrow pointing from the denominator of the coupling coefficient formula to the dot product term $\mathbf{u}_{j|i}^T \mathbf{v}_j$ in the bias update formula. Another yellow arrow points from the dot product term to the bias b_{ij} . A third yellow arrow points from the bias b_{ij} to the numerator of the coupling coefficient formula. The dot product term $\mathbf{u}_{j|i}^T \mathbf{v}_j$ is circled in yellow in the bias update formula.

* The coefficients

- ✓ The dot product measures the agreement of the layer output at j with the prediction made only based on i
- ✓ Depend on the location and type of the capsule

Neurons versus Capsules

Capsule vs. Traditional Neuron			
Input from low-level capsule/neuron	<u>vector($\mathbf{u}_i$)</u>	scalar(x_i)	
Operation	Affine Transform	$\hat{\mathbf{u}}_{j i} = \mathbf{W}_{ij} \mathbf{u}_i$	—
	Weighting	$\mathbf{s}_j = \sum_i c_{ij} \hat{\mathbf{u}}_{j i}$	$a_j = \sum_i w_i x_i + b$
	Sum		
	<u>Nonlinear Activation</u>	$\mathbf{v}_j = \frac{\ \mathbf{s}_j\ ^2}{1 + \ \mathbf{s}_j\ ^2} \frac{\mathbf{s}_j}{\ \mathbf{s}_j\ }$	$h_j = f(a_j)$
Output	<u>vector($\mathbf{v}_j$)</u>	scalar(h_j)	

Routing algorithm

η -parameters

Procedure 1 Routing algorithm

```
1: procedure ROUTING( $\hat{\mathbf{u}}_{j|i}, r, l$ )
2:   for all capsule  $i$  in layer  $l$  and capsule  $j$  in layer  $(l + 1)$ :  $b_{ij} \leftarrow 0$ .
3:   for  $r$  iterations do
4:     for all capsule  $i$  in layer  $l$ :  $\mathbf{c}_i \leftarrow \text{softmax}(\mathbf{b}_i)$ 
5:     for all capsule  $j$  in layer  $(l + 1)$ :  $\mathbf{s}_j \leftarrow \sum_i c_{ij} \hat{\mathbf{u}}_{j|i}$ 
6:     for all capsule  $j$  in layer  $(l + 1)$ :  $\mathbf{v}_j \leftarrow \text{squash}(\mathbf{s}_j)$ 
7:     for all capsule  $i$  in layer  $l$  and capsule  $j$  in layer  $(l + 1)$ :  $b_{ij} \leftarrow b_{ij} + \hat{\mathbf{u}}_{j|i} \cdot \mathbf{v}_j$ 
   return  $\mathbf{v}_j$ 
```

η -routing

$\triangleright$ softmax computes Eq. 3

$\triangleright$ squash computes Eq. 1



Capsule networks - final layer

* Final layer consists of C output capsules

→ The length of the instantiation vectors indicates the probability of the class for the given input.

* Loss function (Margin loss)

$$e_k = \underline{y_k} \max(0, m^+ - ||\mathbf{v}_k^L||^2) + \lambda(1 - y_k) \max(0, ||\mathbf{v}_k^L||^2 - m^-)$$

* Total loss

$$E = \sum_{k=1}^C e_k$$

$$m^+ = 1$$
$$m^- = 0$$

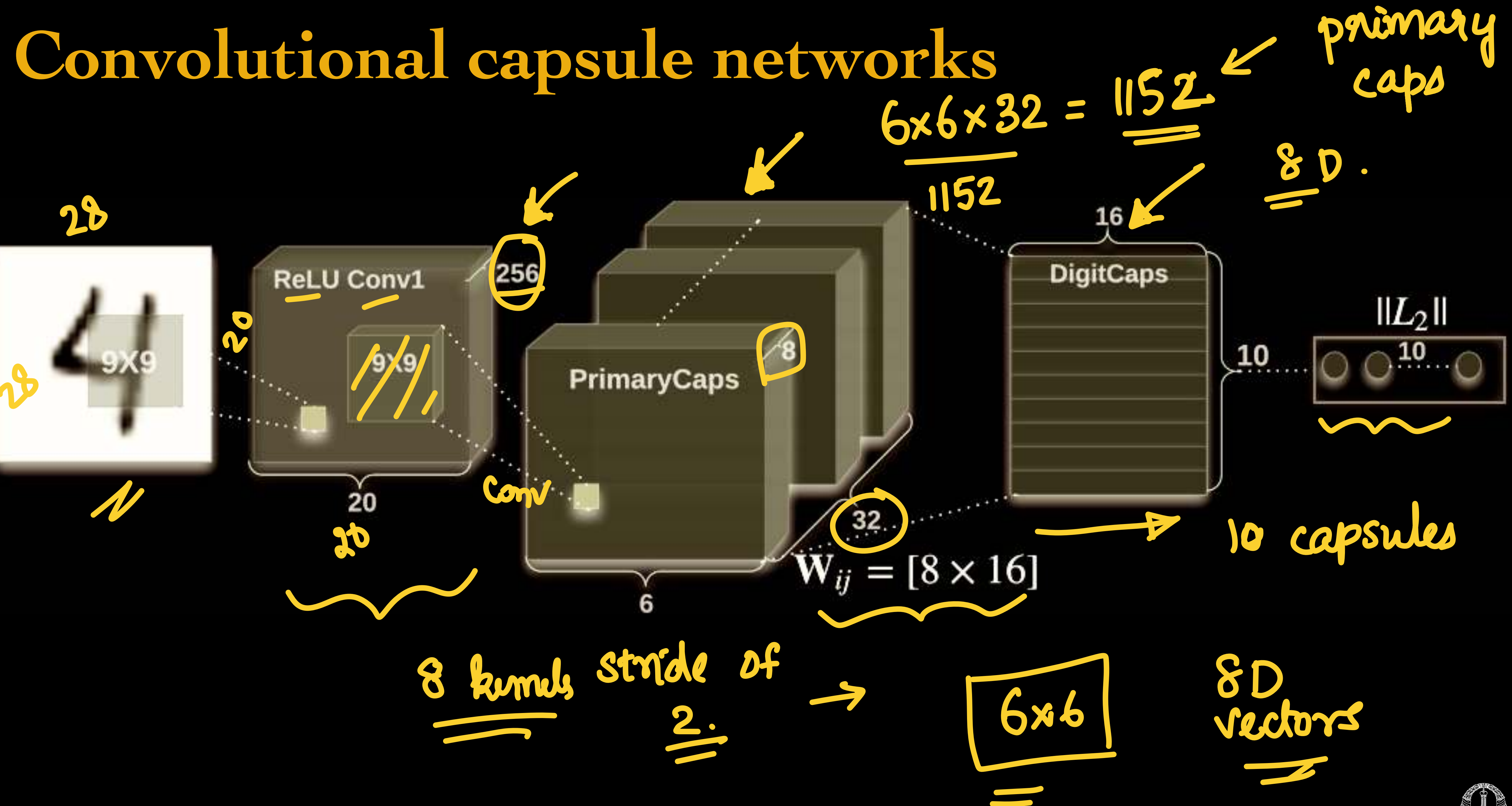
$y_k \rightarrow 1$ if x belongs to k
 $y_k \rightarrow 0$ otherwise

$$\begin{bmatrix} \vdots \\ ||\mathbf{v}_k||^2 \end{bmatrix} 0 \leq ||\mathbf{v}_k||^2 \leq 1$$

C classes = 10

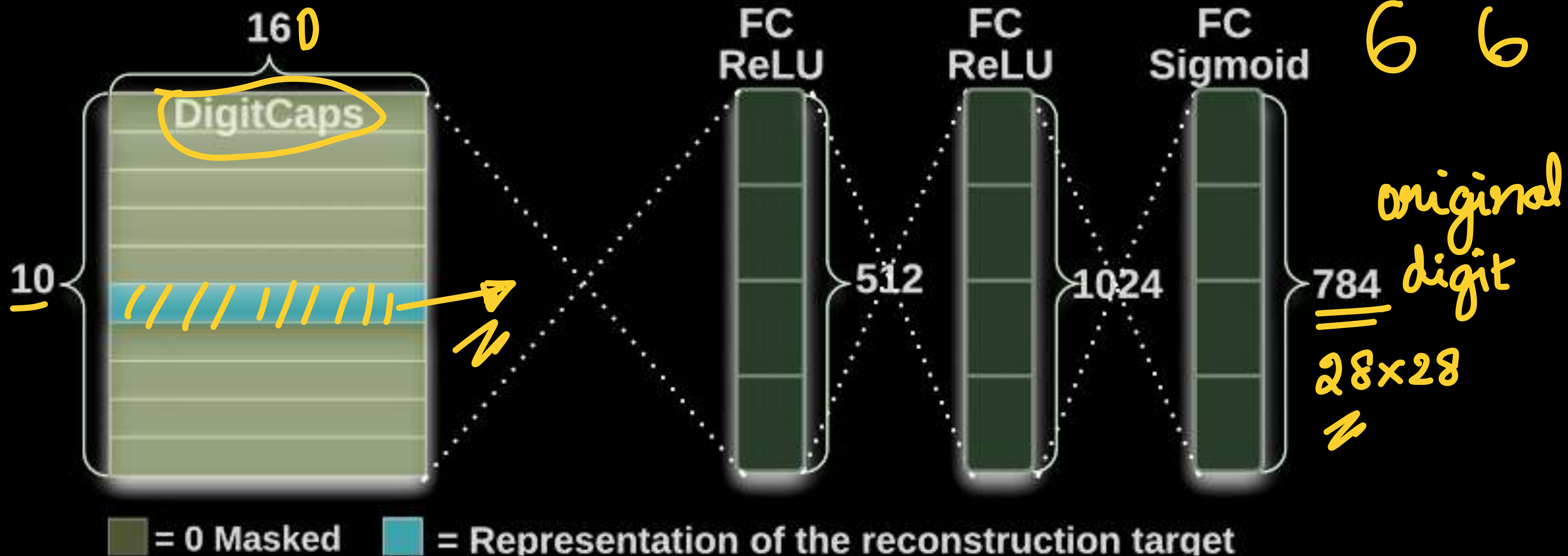


Convolutional capsule networks



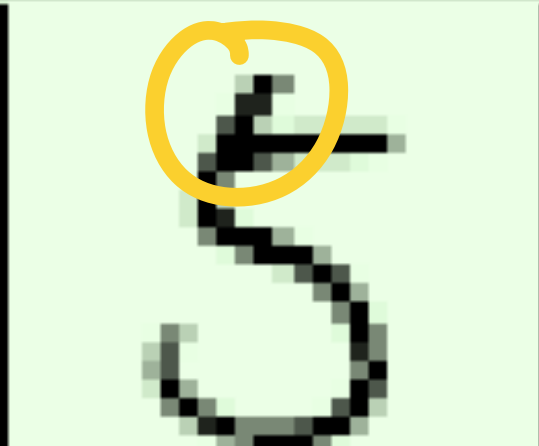
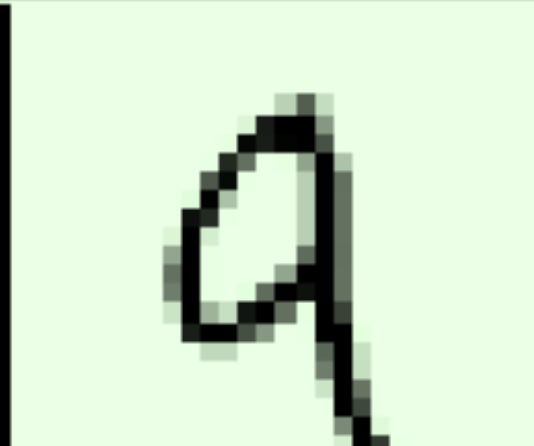
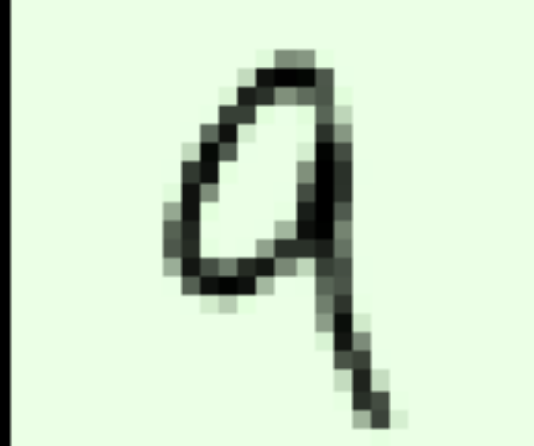
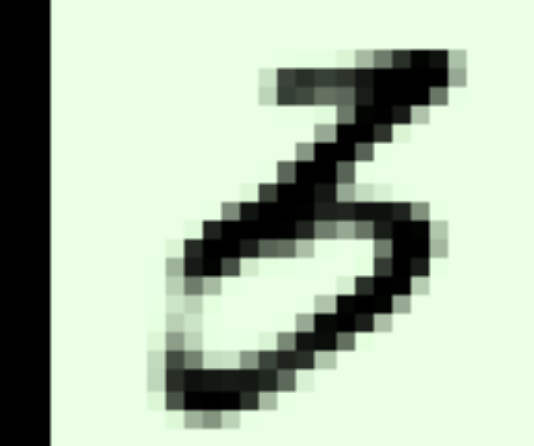
Reconstruction as a regularization loss

- * Use the final capsules to reconstruct the digit images



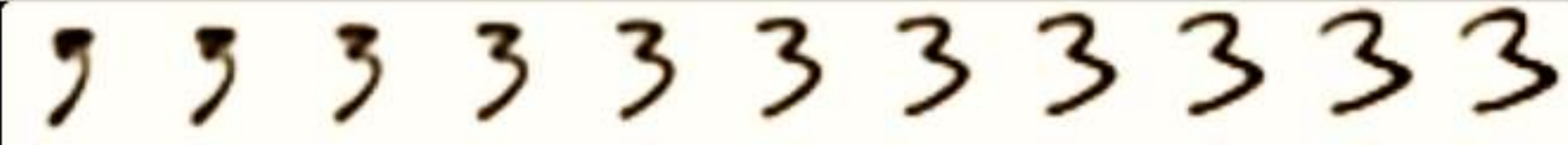
Capsule networks

Figure 3: Sample MNIST test reconstructions of a CapsNet with 3 routing iterations. (l, p, r) represents the label, the prediction and the reconstruction target respectively. The two rightmost columns show two reconstructions of a failure example and it explains how the model confuses a 5 and a 3 in this image. The other columns are from correct classifications and shows that model preserves many of the details while smoothing the noise.

(l, p, r)	$(2, 2, 2)$	$(5, 5, 5)$	$(8, 8, 8)$	$(9, 9, 9)$	$(5, 3, 5)$	$(5, 3, 3)$
Input						
Output						

Understanding the capsule output

Figure 4: Dimension perturbations. Each row shows the reconstruction when one of the 16 dimensions in the DigitCaps representation is tweaked by intervals of 0.05 in the range $[-0.25, 0.25]$.

Scale and thickness	
Localized part	
Stroke thickness	
Localized skew	
Width and translation	
Localized part	

Capsule network performance

Table 1: CapsNet classification test accuracy. The MNIST average and standard deviation results are reported from 3 trials.

Method	Routing	Reconstruction	MNIST (%)
Baseline	-	-	0.39
CapsNet	1	no	$0.34_{\pm 0.032}$
CapsNet	1	yes	$0.29_{\pm 0.011}$
CapsNet	2	yes	$0.25_{\pm 0.005}$

